

Research Report: ROS2 Lifecycle Node & Ackermann Steering Kinematics

May 2026

Contents

1	Theoretical Background	3
1.1	Lifecycle Node in ROS2	3
1.1.1	State Machine	3
1.2	Ackermann Steering Kinematics	3
1.2.1	Bicycle Model	3
1.2.2	Turning Radius	4
1.2.3	Individual Wheel Steering Angles	4
1.2.4	Forward Kinematics	4
1.3	Ackermann Odometry	5
1.4	TF2 and Coordinate Frames	5
2	Labs	5
2.1	Lab 1 – Hello Lifecycle Node	5
2.1.1	Objective	5
2.1.2	Implementation	5
2.1.3	CLI Verification	7
2.1.4	Discussion	8
2.2	Lab 2 – Ackermann Kinematics Calculator	9
2.2.1	Objective	9
2.2.2	Implementation	9
2.2.3	Unit Test Results	10
2.3	Lab 3 – Ackermann Lifecycle Node	11
2.3.1	Objective	11
2.3.2	Key Design Decisions	11
2.3.3	Verification	15
2.3.4	Discussion	16
2.4	Lab 4 – Visualize Path and Odometry	16
2.4.1	Objective	16
2.4.2	Discussion	16
3	Homework Assignments	17
3.1	Homework 1 – Lifecycle Manager	17
3.1.1	Objective	17
3.1.2	Key Implementation Detail: Hidden Client Node	17
3.1.3	Start and Stop Sequences	18
3.1.4	Verification	19
3.2	Homework 2 – Ackermann vs Differential Drive	19
3.2.1	Objective	19
3.2.2	Why the Trajectories Look the Same	19
3.2.3	Simulation Scenario	20
3.2.4	Discussion	20
3.3	Homework 3 – Ackermann Odometry with Noise	20
3.3.1	Objective	20
3.3.2	Noise Model	20
3.3.3	Implementation	21
3.3.4	Predicted Results	23

3.3.5 Discussion	23
4 References	24

Theoretical Background

1.1. Lifecycle Node in ROS2

A standard `rclcpp::Node` starts running immediately when created and has no built-in way to pause or restart safely. This is fine for simple programs, but in a real robot system — where many nodes must start in a specific order and stop cleanly — it causes problems.

A **Lifecycle Node** (`rclcpp_lifecycle::LifecycleNode`) solves this by adding a *state machine* to every node. The system can now control each node's lifecycle explicitly, ensuring predictable startup and shutdown.

1.1.1. State Machine

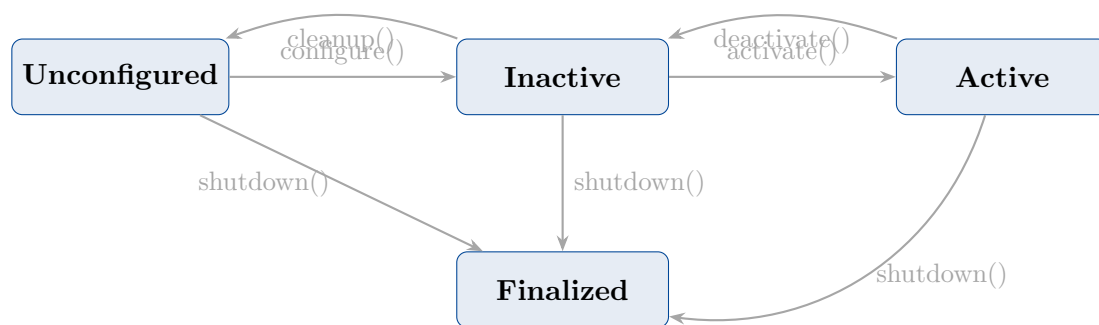


Figure 1: Lifecycle Node state machine

Each transition maps to a callback function that the developer overrides:

Callback	Typical usage
<code>on_configure()</code>	Read parameters, create publishers/subscribers/timers
<code>on_activate()</code>	Activate publishers, start timers, reset state
<code>on_deactivate()</code>	Deactivate publishers, cancel timers
<code>on_cleanup()</code>	Destroy all resources (reset smart pointers)
<code>on_shutdown()</code>	Final cleanup before node exits

A **Lifecycle Publisher** only sends messages when the node is in the *Active* state. Calling `publish()` while inactive is silently ignored. This prevents stale or unwanted data from reaching other nodes during initialization.

1.2. Ackermann Steering Kinematics

Ackermann steering is the geometry used by most cars and wheeled robots with front-wheel steering. Its key property is that all four wheels rotate around a single instantaneous center point — meaning no wheel needs to slip sideways during a turn.

1.2.1. Bicycle Model

For kinematics analysis, a four-wheel vehicle is simplified into a two-wheel **bicycle model**: one front wheel and one rear wheel separated by the wheelbase l .

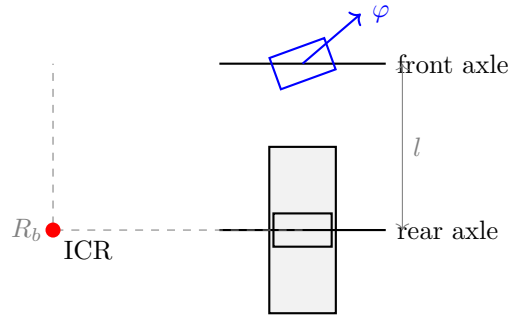


Figure 2: Bicycle model with turning radius R_b and steering angle φ

1.2.2. Turning Radius

Given the center steering angle φ and wheelbase l :

$$R_b = \frac{l}{\tan(\varphi)} \quad (1)$$

When $\varphi \rightarrow 0$, $R_b \rightarrow \infty$ (straight line). When $\varphi \rightarrow \pm 90^\circ$, $R_b \rightarrow 0$, which is physically impossible for a car — this is the main kinematic constraint of Ackermann steering.

1.2.3. Individual Wheel Steering Angles

In a real vehicle, the left and right wheels have different steering angles to satisfy the Ackermann condition (all wheels share the same ICR):

$$\varphi_{\text{left}} = \arctan\left(\frac{l}{R_b - \frac{T}{2}}\right) \quad (2)$$

$$\varphi_{\text{right}} = \arctan\left(\frac{l}{R_b + \frac{T}{2}}\right) \quad (3)$$

where T is the front track width. When turning left ($\varphi > 0$): $\varphi_{\text{left}} > \varphi_{\text{right}}$ because the inner wheel traces a tighter arc.

1.2.4. Forward Kinematics

Given speed v and center steering angle φ , the yaw rate is:

$$\omega = \frac{v}{l} \tan(\varphi) \quad (4)$$

The full state update (Euler integration at timestep Δt):

$$\dot{x} = v \cos(\theta) \quad (5)$$

$$\dot{y} = v \sin(\theta) \quad (6)$$

$$\dot{\theta} = \omega \quad (7)$$

1.3. Ackermann Odometry

Odometry estimates the robot's pose by integrating encoder readings over time. For Ackermann robots, two encoders are needed:

- **Velocity encoder:** measures wheel speed $\rightarrow$ gives v
- **Steering encoder:** measures steering angle $\rightarrow$ gives φ

Euler integration updates position at each timestep:

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{pmatrix} = \begin{pmatrix} x_k + v \cos \theta_k \cdot \Delta t \\ y_k + v \sin \theta_k \cdot \Delta t \\ \theta_k + \omega \cdot \Delta t \end{pmatrix} \quad (8)$$

Odometry drift accumulates over time because every small measurement error in v or φ gets added to the previous estimate. The position error grows without bound — faster with larger noise, and especially when turning.

1.4. TF2 and Coordinate Frames

ROS2 uses a standard frame hierarchy for mobile robots:

odom $\longrightarrow$ base_link $\longrightarrow$ sensor frames

- **odom:** Fixed world frame. The robot's position in this frame is computed by integrating odometry. It is continuous and smooth but drifts over long distances.
- **base_link:** The robot's body frame. Its origin is typically at the center of the rear axle.

The transform odom $\rightarrow$ base_link is published by the odometry node using `tf2_ros::TransformBroadcaster`.

Labs

2.1. Lab 1 – Hello Lifecycle Node

2.1.1. Objective

Write a minimal Lifecycle Node, observe its state machine via CLI, and publish a `/status` message only when active.

2.1.2. Implementation

The node inherits from `rclcpp_lifecycle::LifecycleNode` and overrides all five transition callbacks. A lifecycle publisher is created in `on_configure()` and activated/deactivated with the node state.

Listing 1: MinimalLifecycleNode

```

1 #include <chrono>
2 #include "rclcpp/rclcpp.hpp"
3 #include "rclcpp_lifecycle/lifecycle_node.hpp"
4 #include "rclcpp_lifecycle/lifecycle_publisher.hpp"
5 #include "std_msgs/msg/string.hpp"

```

```

6
7 using namespace std::chrono_literals;
8 using CallbackReturn = rclcpp_lifecycle::node_interfaces::
    LifecycleNodeInterface::CallbackReturn;
9
10 class MinimalLifecycleNode : public rclcpp_lifecycle::
    LifecycleNode
11 {
12 public:
13     MinimalLifecycleNode() : LifecycleNode("minimal_lifecycle_node"
        ) {}
14
15     CallbackReturn on_configure(const rclcpp_lifecycle::State &)
16     {
17         RCLCPP_INFO(get_logger(), "Configuring");
18         pub_ = this->create_publisher<std_msgs::msg::String>("/status
            ", rclcpp::QoS(10));
19         timer_ = create_wall_timer(1s, [this]() { publish_status();
            });
20         timer_>cancel();
21         return CallbackReturn::SUCCESS;
22     }
23
24     CallbackReturn on_activate(const rclcpp_lifecycle::State &
        state)
25     {
26         RCLCPP_INFO(get_logger(), "Activating");
27         // B t b u c g i parent c h u y n state n g
        c c h
28         LifecycleNode::on_activate(state);
29         pub_>on_activate();
30
31         timer_ = create_wall_timer(1s, [this]() { publish_status();
            });
32         return CallbackReturn::SUCCESS;
33     }
34
35     CallbackReturn on_deactivate(const rclcpp_lifecycle::State &
        state)
36     {
37         RCLCPP_INFO(get_logger(), "Deactivating");
38         LifecycleNode::on_deactivate(state);
39         pub_>on_deactivate();
40         timer_>cancel();
41         return CallbackReturn::SUCCESS;
42     }
43
44     CallbackReturn on_cleanup(const rclcpp_lifecycle::State &)
45     {
46         RCLCPP_INFO(get_logger(), "Cleaning up");
47         count_ = 0;

```

```

48     pub_.reset();
49     timer_.reset();
50     return CallbackReturn::SUCCESS;
51 }
52
53 CallbackReturn on_shutdown(const rclcpp_lifecycle::State &)
54 {
55     RCLCPP_INFO(get_logger(), "Shutting down");
56     return CallbackReturn::SUCCESS;
57 }
58
59 private:
60 void publish_status()
61 {
62     if (!pub_->is_activated()) return;
63     auto msg = std_msgs::msg::String();
64     msg.data = "Node is active - " + std::to_string(count_++);
65     RCLCPP_INFO(get_logger(), "Publishing: %s", msg.data.c_str());
66     ;
67     pub_->publish(msg);
68 }
69
70 rclcpp_lifecycle::LifecyclePublisher<std_msgs::msg::String>::
71     SharedPtr pub_;
72 rclcpp::TimerBase::SharedPtr timer_;
73 int count_ = 0;
74 };
75
76 int main(int argc, char ** argv)
77 {
78     rclcpp::init(argc, argv);
79     auto node = std::make_shared<MinimalLifecycleNode>();
80     rclcpp::executors::SingleThreadedExecutor exe;
81     exe.add_node(node->get_node_base_interface());
82     exe.spin();
83     rclcpp::shutdown();
84     return 0;
85 }

```

2.1.3. CLI Verification

Listing 2: Lifecycle CLI commands

```

1 ros2 lifecycle get /minimal_lifecycle_node      # -> unconfigured
2 ros2 lifecycle set /minimal_lifecycle_node configure
3 ros2 lifecycle set /minimal_lifecycle_node activate
4 ros2 topic echo /status

```



```
Summary: 1 package finished [4.99s]
ngxwang@LAPTOP-SS0001C1:~/ros2_ws$ ros2 node list
ngxwang@LAPTOP-SS0001C1:~/ros2_ws$ source install/setup.bash
ngxwang@LAPTOP-SS0001C1:~/ros2_ws$ ros2 node list
ngxwang@LAPTOP-SS0001C1:~/ros2_ws$ ros2 run lifecycle_demo minimal_lifecycle_node
[INFO] [1778663426.292949956] [minimal_lifecycle_node]: Configuring
[INFO] [1778663436.593302116] [minimal_lifecycle_node]: Activating
[INFO] [1778663437.593609380] [minimal_lifecycle_node]: Publishing: Node is active - 0
[INFO] [1778663438.593503618] [minimal_lifecycle_node]: Publishing: Node is active - 1
[INFO] [1778663439.528147362] [minimal_lifecycle_node]: Publishing: Node is active - 2
[INFO] [1778663440.457467962] [minimal_lifecycle_node]: Publishing: Node is active - 3
[INFO] [1778663441.387509494] [minimal_lifecycle_node]: Publishing: Node is active - 4
[INFO] [1778663442.363212961] [minimal_lifecycle_node]: Publishing: Node is active - 5
[INFO] [1778663443.363228588] [minimal_lifecycle_node]: Publishing: Node is active - 6

ngxwang@LAPTOP-SS0001C1: $ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

# Kiểm tra node đã chạy chưa
ros2 lifecycle nodes
/minimal_lifecycle_node
ngxwang@LAPTOP-SS0001C1: $ ros2 lifecycle set /minimal_lifecycle_node configure
Transitioning successful
ngxwang@LAPTOP-SS0001C1: $ ros2 lifecycle set /minimal_lifecycle_node activate
Transitioning successful
ngxwang@LAPTOP-SS0001C1: $
```

Figure 3: state transitions

```
[INFO] [1778663457.139257811] [minimal_lifecycle_node]: Publishing: Node is active - 20
[INFO] [1778663458.115557414] [minimal_lifecycle_node]: Publishing: Node is active - 21
[INFO] [1778663459.910849644] [minimal_lifecycle_node]: Publishing: Node is active - 22
[INFO] [1778663460.901583247] [minimal_lifecycle_node]: Publishing: Node is active - 23
[INFO] [1778663461.901876118] [minimal_lifecycle_node]: Publishing: Node is active - 24
[INFO] [1778663462.901990885] [minimal_lifecycle_node]: Publishing: Node is active - 25
[INFO] [1778663463.873340830] [minimal_lifecycle_node]: Publishing: Node is active - 26
[INFO] [1778663464.784978317] [minimal_lifecycle_node]: Publishing: Node is active - 27
[INFO] [1778663465.702092648] [minimal_lifecycle_node]: Publishing: Node is active - 28
[INFO] [1778663466.626512032] [minimal_lifecycle_node]: Publishing: Node is active - 29
[INFO] [1778663467.626728055] [minimal_lifecycle_node]: Publishing: Node is active - 30
[INFO] [1778663468.626697402] [minimal_lifecycle_node]: Publishing: Node is active - 31

ngxwang@LAPTOP-SS0001C1: $ ros2 lifecycle set /minimal_lifecycle_node configure
Transitioning successful
ngxwang@LAPTOP-SS0001C1: $ ros2 lifecycle set /minimal_lifecycle_node activate
Transitioning successful
ngxwang@LAPTOP-SS0001C1: $ ros2 topic echo /status
data: Node is active - 24
...
data: Node is active - 25
...
data: Node is active - 26
...
data: Node is active - 27
...
data: Node is active - 28
...
data: Node is active - 29
...
data: Node is active - 30
...
data: Node is active - 31
...
```

Figure 4: /status messages

2.1.4. Discussion

Q: What happens if `on_configure()` returns `FAILURE`?

The node stays in the `Unconfigured` state. ROS2 does not move it to `Inactive`, and any Lifecycle Manager watching the node will receive a failure signal. The node is still alive and can be retried.

Q: Can a Lifecycle Publisher publish while the node is Inactive?

No. A Lifecycle Publisher's `publish()` call is silently ignored when the publisher has not been activated. This is intentional — it prevents data from flowing before the system is ready.

Q: Should a subscriber callback process data when the node is inactive?

It should not. The best practice is to check the current state at the start of every callback and return early if the node is not `Active`. This prevents the node from acting on commands it has not been told to handle yet.

2.2. Lab 2 – Ackermann Kinematics Calculator*2.2.1. Objective*

Implement a standalone `AckermannKinematics` class in C++ and verify it with unit tests.

2.2.2. Implementation

Listing 3: AckermannKinematics class

```

1 #pragma once
2 #include <cmath>
3 #include <limits>
4
5 class AckermannKinematics
6 {
7 public:
8     AckermannKinematics(double wheelbase, double track_width_front)
9         : L_(wheelbase), T_(track_width_front) {}
10
11     double turningRadius(double phi) const
12     {
13         if (std::abs(phi) < 1e-9)
14             return std::numeric_limits<double>::infinity();
15         return L_ / std::tan(phi);
16     }
17
18     double computeLeftSteeringAngle(double phi) const
19     {
20         double R = turningRadius(phi);
21         if (std::isinf(R)) return 0.0;
22         return std::atan(L_ / (R - T_ / 2.0));
23     }
24
25     double computeRightSteeringAngle(double phi) const
26     {
27         double R = turningRadius(phi);
28         if (std::isinf(R)) return 0.0;
29         return std::atan(L_ / (R + T_ / 2.0));
30     }
31

```

```

32  double computeCentralAngle(double phi_left, double phi_right)
    const
33  {
34      return (phi_left + phi_right) / 2.0;
35  }
36
37  double wheelbase() const { return L_; }
38
39 private:
40  double L_; // wheelbase
41  double T_; // track width front
42  };

```

2.2.3. Unit Test Results

Test case	Input φ	Expected	Result
Straight	0	$\varphi_L = \varphi_R = 0, R = \infty$	PASS
Left turn	0.3 rad	$\varphi_L > \varphi_R$	PASS
Symmetry	φ vs $-\varphi$	$\varphi_L(\varphi) = \varphi_R(-\varphi)$	PASS
Roundtrip	any φ	$\text{computeCentralAngle}(\varphi_L, \varphi_R) \approx \varphi$	PASS

Listing 4: Unit Test

```

1  #include <gtest/gtest.h>
2  #include <cmath>
3  #include "lifecycle_demo/ackermann_kinematics.hpp"
4
5  class KinematicsTest : public ::testing::Test {
6  protected:
7      AckermannKinematics kin{1.5, 1.0}; // wheelbase=1.5m, track
        =1.0m
8  };
9
10 TEST_F(KinematicsTest, ZeroAngle) {
11     EXPECT_DOUBLE_EQ(kin.computeLeftSteeringAngle(0.0), 0.0);
12     EXPECT_DOUBLE_EQ(kin.computeRightSteeringAngle(0.0), 0.0);
13     EXPECT_TRUE(std::isinf(kin.turningRadius(0.0)));
14 }
15
16 TEST_F(KinematicsTest, PositiveAngle) {
17     double phi = 0.3; // rad
18     double left = kin.computeLeftSteeringAngle(phi);
19     double right = kin.computeRightSteeringAngle(phi);
20
21     EXPECT_GT(left, right);
22     EXPECT_GT(left, 0.0);
23     EXPECT_GT(right, 0.0);
24 }
25
26 TEST_F(KinematicsTest, Symmetry) {
27     double phi = 0.3;

```

```

28
29     EXPECT_NEAR(kin.computeLeftSteeringAngle(phi),
30                 -kin.computeRightSteeringAngle(-phi), 1e-9);
31 }
32
33 int main(int argc, char **argv) {
34     ::testing::InitGoogleTest(&argc, argv);
35     return RUN_ALL_TESTS();
36 }

```

```

ngxwang@LAPTOP-SS0001C1:~/ros2_ws/src$ ./build/lifecycle_demo/test_kinematics
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from KinematicsTest
[ RUN     ] KinematicsTest.ZeroAngle
[ OK      ] KinematicsTest.ZeroAngle (0 ms)
[ RUN     ] KinematicsTest.PositiveAngle
[ OK      ] KinematicsTest.PositiveAngle (1 ms)
[ RUN     ] KinematicsTest.Symmetry
[ OK      ] KinematicsTest.Symmetry (0 ms)
[-----] 3 tests from KinematicsTest (1 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (1 ms total)
[ PASSED ] 3 tests.
ngxwang@LAPTOP-SS0001C1:~/ros2_ws/src$

```

Figure 5: /status messages

2.3. Lab 3 – Ackermann Lifecycle Node

2.3.1. Objective

Combine the Lifecycle Node pattern with Ackermann kinematics to produce a node that subscribes to `/cmd_ackermann` and publishes `/odom`, `/wheel_commands`, `/path`, and the TF `odom → base_link`.

2.3.2. Key Design Decisions

- All publishers, the subscriber, and the timer are created in `on_configure()` — *not* in the constructor. This allows clean re-configuration.
- The timer is created *once* in `on_configure()` and immediately cancelled. `on_activate()` calls `timer_>reset()` to resume it; `on_deactivate()` calls `timer_>cancel()` to pause it. Creating a new timer in `on_activate()` is a common mistake that creates orphaned timers.

Listing 5: Ackermann Lifecycle Node

```

1 #include <chrono>
2 #include <cmath>
3 #include "rclcpp/rclcpp.hpp"
4 #include "rclcpp_lifecycle/lifecycle_node.hpp"
5 #include "rclcpp_lifecycle/lifecycle_publisher.hpp"
6 #include "ackermann_msgs/msg/ackermann_drive.hpp"
7 #include "nav_msgs/msg/odometry.hpp"
8 #include "nav_msgs/msg/path.hpp"
9 #include "geometry_msgs/msg/pose_stamped.hpp"
10 #include "tf2_ros/transform_broadcaster.h"

```

```

11 #include "geometry_msgs/msg/transform_stamped.hpp"
12 #include "std_msgs/msg/float64_multi_array.hpp"
13
14 using namespace std::chrono_literals;
15 using CallbackReturn = rclcpp_lifecycle::node_interfaces::
    LifecycleNodeInterface::CallbackReturn;
16
17 class AckermannKinematicsNode : public rclcpp_lifecycle::
    LifecycleNode
18 {
19 public:
20     AckermannKinematicsNode() : LifecycleNode("
        ackermann_kinematics_node")
21     {
22         declare_parameter("wheelbase", 1.5);
23         declare_parameter("track_width_front", 1.0);
24     }
25
26     CallbackReturn on_configure(const rclcpp_lifecycle::State &)
27     {
28         wheelbase_ = get_parameter("wheelbase").as_double();
29         track_width_ = get_parameter("track_width_front").as_double()
        ;
30
31
32         odom_pub_ = this->create_publisher<nav_msgs::msg::Odometry>("/
            odom", rclcpp::QoS(10));
33         path_pub_ = this->create_publisher<nav_msgs::msg::Path>("/
            path", rclcpp::QoS(10));
34         wheel_pub_ = this->create_publisher<std_msgs::msg::
            Float64MultiArray>("/wheel_commands", rclcpp::QoS(10));
35
36         tf_broadcaster_ = std::make_unique<tf2_ros::
            TransformBroadcaster>(*this);
37
38         cmd_sub_ = create_subscription<ackermann_msgs::msg::
            AckermannDrive>(
39             "/cmd_ackermann", 10,
40             [this](const ackermann_msgs::msg::AckermannDrive::SharedPtr
                msg) {
41                 cmd_callback(msg);
42             });
43
44         timer_ = create_wall_timer(50ms, [this]() { update(); });
45         timer_->cancel();
46
47         path_msg_.header.frame_id = "odom";
48         RCLCPP_INFO(get_logger(), "Configured. wheelbase=%.2f, track
            =%.2f", wheelbase_, track_width_);
49         return CallbackReturn::SUCCESS;
50     }

```

```

51
52 CallbackReturn on_activate(const rclcpp_lifecycle::State &
53     state)
54 {
55     LifecycleNode::on_activate(state);
56     odom_pub_ -> on_activate();
57     path_pub_ -> on_activate();
58     wheel_pub_ -> on_activate();
59
60     path_msg_.poses.clear();
61     x_ = 0; y_ = 0; theta_ = 0;
62
63     timer_ = create_wall_timer(50ms, [this]() { update(); });
64     timer_ -> reset();
65     RCLCPP_INFO(get_logger(), "Activated.");
66     return CallbackReturn::SUCCESS;
67 }
68
69 CallbackReturn on_deactivate(const rclcpp_lifecycle::State &
70     state)
71 {
72     LifecycleNode::on_deactivate(state);
73     odom_pub_ -> on_deactivate();
74     path_pub_ -> on_deactivate();
75     wheel_pub_ -> on_deactivate();
76     timer_ -> cancel();
77     RCLCPP_INFO(get_logger(), "Deactivated.");
78     return CallbackReturn::SUCCESS;
79 }
80
81 CallbackReturn on_cleanup(const rclcpp_lifecycle::State &)
82 {
83     odom_pub_.reset();
84     path_pub_.reset();
85     cmd_sub_.reset();
86     timer_.reset();
87     wheel_pub_.reset();
88     tf_broadcaster_.reset();
89     RCLCPP_INFO(get_logger(), "Cleaned up.");
90     return CallbackReturn::SUCCESS;
91 }
92
93 CallbackReturn on_shutdown(const rclcpp_lifecycle::State &) {
94     return CallbackReturn::SUCCESS; }
95
96 private:
97 void cmd_callback(const ackermann_msgs::msg::AckermannDrive::
98     SharedPtr msg)
99 {
100     current_speed_ = msg->speed;

```

```

98     current_steering_ = msg->steering_angle;
99 }
100
101 void update()
102 {
103     auto now = get_clock()->now();
104     double dt = 0.05;
105
106     double omega = (wheelbase_ > 1e-9)
107         ? (current_speed_ / wheelbase_) * std::tan(
108             current_steering_)
109         : 0.0;
110
111     x_      += current_speed_ * std::cos(theta_) * dt;
112     y_      += current_speed_ * std::sin(theta_) * dt;
113     theta_ += omega * dt;
114
115     double steering_left = 0.0;
116     double steering_right = 0.0;
117
118     if (std::abs(current_steering_) > 1e-4) {
119         double R = wheelbase_ / std::tan(current_steering_);
120
121         steering_left = std::atan(wheelbase_ / (R - track_width_ /
122             2.0));
123         steering_right = std::atan(wheelbase_ / (R + track_width_ /
124             2.0));
125     } else {
126         steering_left = steering_right = current_steering_; // i
127         t h n g
128     }
129
130     // TF: odom -> base_link
131     geometry_msgs::msg::TransformStamped tf;
132     tf.header.stamp = now;
133     tf.header.frame_id = "odom";
134     tf.child_frame_id = "base_link";
135     tf.transform.translation.x = x_;
136     tf.transform.translation.y = y_;
137     tf.transform.rotation.z = std::sin(theta_ / 2.0);
138     tf.transform.rotation.w = std::cos(theta_ / 2.0);
139     tf_broadcaster->sendTransform(tf);
140
141     // Odometry
142     nav_msgs::msg::Odometry odom;
143     odom.header.stamp = now;
144     odom.header.frame_id = "odom";
145     odom.child_frame_id = "base_link";
146     odom.pose.pose.position.x = x_;
147     odom.pose.pose.position.y = y_;
148     odom.pose.pose.orientation = tf.transform.rotation;

```

```

145     odom.twist.twist.linear.x   = current_speed_;
146     odom.twist.twist.angular.z = omega;
147     odom_pub_ -> publish(odom);
148
149     // Path
150     geometry_msgs::msg::PoseStamped ps;
151     ps.header = odom.header;
152     ps.pose   = odom.pose.pose;
153     path_msg_.header.stamp = now;
154     path_msg_.poses.push_back(ps);
155     path_pub_ -> publish(path_msg_);
156
157
158     std_msgs::msg::Float64MultiArray wheel_msg;
159     wheel_msg.data = {current_speed_, steering_left,
160 steering_right};
161     wheel_pub_ -> publish(wheel_msg);
162 }
163
164 double wheelbase_{1.5}, track_width_{1.0};
165 double x_{0}, y_{0}, theta_{0};
166 double current_speed_{0}, current_steering_{0};
167
168 rclcpp_lifecycle::LifecyclePublisher<nav_msgs::msg::Odometry>::
169   SharedPtr odom_pub_;
170 rclcpp_lifecycle::LifecyclePublisher<nav_msgs::msg::Path>::
171   SharedPtr path_pub_;
172 rclcpp_lifecycle::LifecyclePublisher<std_msgs::msg::
173   Float64MultiArray>::SharedPtr wheel_pub_;
174 rclcpp::Subscription<ackermann_msgs::msg::AckermannDrive>::
175   SharedPtr cmd_sub_;
176 rclcpp::TimerBase::SharedPtr timer_;
177 std::unique_ptr<tf2_ros::TransformBroadcaster> tf_broadcaster_;
178 nav_msgs::msg::Path path_msg_;
179 };
180
181 int main(int argc, char ** argv)
182 {
183     rclcpp::init(argc, argv);
184     auto node = std::make_shared<AckermannKinematicsNode>();
185     rclcpp::executors::SingleThreadedExecutor exe;
186     exe.add_node(node->get_node_base_interface());
187     exe.spin();
188     rclcpp::shutdown();
189     return 0;
190 }

```

2.3.3. Verification

```

1 ros2 topic pub /cmd_ackermann ackermann_msgs/msg/AckermannDrive \
2   "{speed: 1.0, steering_angle: 0.3}"

```



```
3 ros2 topic echo /wheel_commands
4 ros2 topic echo /odom
5 rviz2
```

The simulation video is uploaded [here](#).

2.3.4. Discussion

Q: Why is AckermannDrive better than geometry_msgs/Twist?

Twist only carries linear velocity and yaw rate (ω). It loses the physical meaning of the steering angle φ . **AckermannDrive** carries both speed and steering angle directly, which is what a real Ackermann vehicle actuates. This makes it easier to compute individual wheel angles and send realistic commands to hardware.

Q: What is the difference between steering angle φ and yaw rate ω ?

φ is a geometric quantity — the angle the front wheels are turned. ω is a kinematic quantity — the rate at which the robot's heading changes. They are related by $\omega = (v/l) \tan(\varphi)$, so ω depends on both φ and the current speed v .

Q: What happens when $\varphi \rightarrow \pm 90^\circ$?

From equation (1), $R_b = l / \tan(\varphi) \rightarrow 0$. This means the robot would rotate around its own rear axle, which requires infinite lateral force and is physically impossible. Real cars limit steering to about ± 35 – 40° .

2.4. Lab 4 – Visualize Path and Odometry

2.4.1. Objective

Publish `nav_msgs/msg/Path` and visualize the robot's trajectory in RViz2 for three motion patterns: circle, S-curve, and reverse.

The simulation videos are uploaded [here](#) and [here](#).

2.4.2. Discussion

Q: How does odometry drift occur?

Every integration step adds a small error from encoder noise. Because the heading θ error accumulates and affects both $\dot{x}$ and $\dot{y}$, position error grows over time — quadratically in the worst case. Turning amplifies drift because steering encoder noise directly affects ω .

Q: If $\varphi_{\text{left}} = \varphi_{\text{right}}$, is the trajectory still correct Ackermann?

No. Setting both wheels to the same angle violates the Ackermann condition. The inner wheel would need to travel the same arc length as the outer wheel, which means it must slip sideways. In simulation this appears to work because there is no tire slip model, but on a real robot it causes tyre wear and inaccurate odometry.

Homework Assignments

3.1. Homework 1 – Lifecycle Manager

3.1.1. Objective

Write an `AckermannSystemManager` node that exposes `/system/start` and `/system/stop` services to orchestrate the full lifecycle of `AckermannKinematicsNode`.

3.1.2. Key Implementation Detail: Hidden Client Node

Calling `spin_until_future_complete()` on the same node that is currently being spun causes a deadlock. The solution is to create a separate hidden node solely for the service client:

Listing 6: Ackermann System Manager

```

1 #include "rclcpp/rclcpp.hpp"
2 #include "lifecycle_msgs/srv/change_state.hpp"
3 #include "lifecycle_msgs/msg/transition.hpp"
4 #include "std_srvs/srv/trigger.hpp"
5 #include "std_msgs/msg/string.hpp"
6
7 using namespace std::chrono_literals;
8
9 class AckermannSystemManager : public rclcpp::Node {
10 public:
11     AckermannSystemManager() : Node("ackermann_system_manager") {
12         // --- IM QUAN T R NG N H T ---
13         // T o m t node n r i n g b i t Client
14         k h n g b n g E x e c u t o r v i N o d e c h n h
15         client_node_ = rclcpp::Node::make_shared("
16         manager_client_node_hidden");
17         client_ = client_node_>create_client<lifecycle_msgs::srv
18         ::ChangeState>("/ackermann_kinematics_node/change_state");
19
20         start_srv_ = create_service<std_srvs::srv::Trigger>(
21         "/system/start", std::bind(&AckermannSystemManager::
22         handle_start, this, std::placeholders::_1, std::placeholders::_2));
23         stop_srv_ = create_service<std_srvs::srv::Trigger>(
24         "/system/stop", std::bind(&AckermannSystemManager::
25         handle_stop, this, std::placeholders::_1, std::placeholders::_2
26         ));
27
28         status_pub_ = create_publisher<std_msgs::msg::String>("/
29         system/status", 10);
30
31         RCLCPP_INFO(get_logger(), "Manager ready.");
32     }
33
34 private:
35     bool call_transition(uint8_t transition_id) {

```

```

29     if (!client_ -> wait_for_service(2s)) return false;
30     auto req = std::make_shared<lifecycle_msgs::srv::
ChangeState::Request>();
31     req->transition.id = transition_id;
32
33     auto result = client_ -> async_send_request(req);
34
35     // Spin con node N thay v node CH NH
36     if (rclcpp::spin_until_future_complete(client_node_,
result) != rclcpp::FutureReturnCode::SUCCESS) return false;
37     return result.get() -> success;
38 }
39 //...
40 void publish_status(std::string status) {
41     auto msg = std_msgs::msg::String();
42     msg.data = status;
43     status_pub_ -> publish(msg);
44 }
45
46 rclcpp::Node::SharedPtr client_node_; // Khai b o Node n
47 rclcpp::Service<std_srvs::srv::Trigger>::SharedPtr start_srv_
, stop_srv_;
48 rclcpp::Client<lifecycle_msgs::srv::ChangeState>::SharedPtr
client_;
49 rclcpp::Publisher<std_msgs::msg::String>::SharedPtr
status_pub_;
50 };
51 int main(int argc, char** argv) {
52     rclcpp::init(argc, argv);
53     rclcpp::spin(std::make_shared<AckermannSystemManager>());
54     rclcpp::shutdown();
55     return 0;
56 }

```

3.1.3. Start and Stop Sequences

Listing 7: Start sequence with rollback on failure

```

1 void handle_start(const std::shared_ptr<std_srvs::srv::Trigger::
Request>, std::shared_ptr<std_srvs::srv::Trigger::Response> res
) {
2     if (call_transition(lifecycle_msgs::msg::Transition::
TRANSITION_CONFIGURE)) {
3         if (call_transition(lifecycle_msgs::msg::Transition::
TRANSITION_ACTIVATE)) {
4             res->success = true;
5             res->message = "System is ACTIVE";
6             publish_status("ACTIVE");
7             return;
8         }
9         call_transition(lifecycle_msgs::msg::Transition::
TRANSITION_CLEANUP);

```

```

10     }
11     res->success = false;
12     publish_status("ERROR_START_FAILED");
13 }
14
15 void handle_stop(const std::shared_ptr<std_srvs::srv::Trigger::
    Request>, std::shared_ptr<std_srvs::srv::Trigger::Response> res
    ) {
16     call_transition(lifecycle_msgs::msg::Transition::
    TRANSITION_DEACTIVATE);
17     call_transition(lifecycle_msgs::msg::Transition::
    TRANSITION_CLEANUP);
18     res->success = true;
19     publish_status("INACTIVE");
20 }

```

3.1.4. Verification

```

1 ros2 service call /system/start std_srvs/srv/Trigger {}
2 # Response: success=True, message=' H      t h ng      ACTIVE '
3
4 ros2 service call /system/stop std_srvs/srv/Trigger {}
5 # Response: success=True
6
7 ros2 lifecycle get /ackermann_kinematics_node
8 # -> unconfigured

```

The simulation video is uploaded [here](#).

3.2. Homework 2 – Ackermann vs Differential Drive

3.2.1. Objective

Simulate both drive models for 10 seconds using the same input commands, publish their paths to RViz2 in real time, and export a CSV for offline analysis.

3.2.2. Why the Trajectories Look the Same

Both models use the same forward kinematics equations because they share the same mathematical relationship $\omega = (v/l) \tan(\varphi)$. The difference appears in practice:

- A **differential drive** robot can rotate in place (set one wheel forward, one backward). Ackermann cannot.
- An **Ackermann** robot has a minimum turning radius constraint ($R_b \geq l / \tan(\varphi_{\max})$). Differential drive has no such constraint.
- In the presence of **noise and slip**, the two models diverge because differential drive slip is symmetric while Ackermann slip is asymmetric between inner and outer wheels.

3.2.3. Simulation Scenario

Time (s)	Speed v (m/s)	Steering φ (rad)
0–2	1.5	0.0 (straight)
2–5	1.5	0.3 (left turn)
5–7	1.5	0.0 (straight)
7–10	1.5	−0.3 (right turn)

The simulation is uploaded [here](#).

3.2.4. Discussion

Q: Are the trajectories the same?

Yes, in ideal conditions they are mathematically identical. Both use the same $\omega = (v/l) \tan(\varphi)$ relationship. Differences emerge only with noise, tire slip, or different motion primitives (e.g., in-place rotation).

Q: What constraints does Ackermann have?

The steering angle is limited: $|\varphi| \leq \varphi_{\max}$, which gives a minimum turning radius $R_{\min} = l / \tan(\varphi_{\max})$. The robot cannot rotate in place. Each wheel must be steered to a slightly different angle ($\varphi_L \neq \varphi_R$) for correct geometry.

Q: What is the typical steering angle limit in practice?

Most passenger cars limit steering to approximately $\pm 35^\circ$ (≈ 0.61 rad). Racing and off-road vehicles may reach $\pm 45^\circ$. Small robot platforms like the Ackermann RC cars used in research typically use $\pm 30^\circ$.

3.3. Homework 3 – Ackermann Odometry with Noise

3.3.1. Objective

Add Gaussian noise to the velocity and steering encoders, simulate the same 10-second scenario at three noise levels, and compare each noisy odometry estimate against the ground truth.

3.3.2. Noise Model

At each timestep, independent Gaussian samples are added to the encoder readings:

$$v_{\text{noisy}} = v + \mathcal{N}(0, \sigma^2) \quad (9)$$

$$\varphi_{\text{noisy}} = \varphi + \mathcal{N}(0, \sigma^2) \quad (10)$$

These noisy values are then integrated with the same Euler equations. The seed is fixed (`std::mt19937 rng(42)`) so results are reproducible.

3.3.3. Implementation

Listing 8: Noise injection in odometry update

```

1 #include <chrono>
2 #include <cmath>
3 #include <random> // Th      v i n      t o      n h i u
4
5 #include "rclcpp/rclcpp.hpp"
6 #include "nav_msgs/msg/path.hpp"
7 #include "geometry_msgs/msg/pose_stamped.hpp"
8
9 using namespace std::chrono_literals;
10
11 struct RobotState { double x{0}, y{0}, theta{0}; };
12
13 class AckermannNoiseNode : public rclcpp::Node
14 {
15 public:
16     AckermannNoiseNode() : Node("ackermann_noise_node")
17     {
18         // Khai b o parameter      d      d n g test c c m c
19         n h i u (0.001, 0.01, 0.05)
20         this->declare_parameter("noise_level", 0.001);
21         noise_level_ = this->get_parameter("noise_level").
22         as_double();
23
24         truth_path_pub_ = create_publisher<nav_msgs::msg::Path>("/path/ground_truth", 10);
25         noisy_path_pub_ = create_publisher<nav_msgs::msg::Path>("/path/noisy_odom", 10);
26
27         truth_path_.header.frame_id = "odom";
28         noisy_path_.header.frame_id = "odom";
29
30         // K h i      t o      b      sinh n h i u Gaussian
31         generator_ = std::default_random_engine(std::
32         random_device{}());
33         noise_dist_ = std::normal_distribution<double>(0.0,
34         noise_level_);
35
36         timer_ = create_wall_timer(50ms, [this]() { update(); });
37
38         RCLCPP_INFO(get_logger(), "Starting Noise Simulation with
39         std_dev = %.3f", noise_level_);
40     }
41 private:
42     void update()
43     {
44         if (sim_time_ > 15.0) {
45             RCLCPP_INFO_ONCE(get_logger(), "Simulation Completed.
46             ");

```

```

42         timer_ ->cancel();
43         return;
44     }
45
46     // 1. Lnh iu khin l t ng (V d :
47     // ch y v ng tr n)
48     double cmd_v = 1.0;
49     double cmd_phi = 0.2;
50
51     // 2. C p n h t Ground Truth (Kh ng n h i u )
52     truth_state_ = updateKinematics(truth_state_, cmd_v,
53     cmd_phi, dt_);
54
55     // 3. T o t n h i u c m b i n t h c t b
56     // n h i u (Th m Gaussian noise)
57     double noisy_v = cmd_v + noise_dist_(generator_);
58     double noisy_phi = cmd_phi + noise_dist_(generator_);
59
60     // 4. C p n h t Odometry (D a tr n t n h i u
61     // n h i u )
62     noisy_state_ = updateKinematics(noisy_state_, noisy_v,
63     noisy_phi, dt_);
64
65     auto now = get_clock()->now();
66     appendPath(truth_path_, truth_state_, now);
67     appendPath(noisy_path_, noisy_state_, now);
68
69     truth_path_pub_ ->publish(truth_path_);
70     noisy_path_pub_ ->publish(noisy_path_);
71
72     sim_time_ += dt_;
73 }
74
75 RobotState updateKinematics(RobotState s, double v, double
76 phi, double dt)
77 {
78     double omega = (v / wheelbase_) * std::tan(phi);
79     s.x += v * std::cos(s.theta) * dt;
80     s.y += v * std::sin(s.theta) * dt;
81     s.theta += omega * dt;
82     return s;
83 }
84
85 void appendPath(nav_msgs::msg::Path & path, const RobotState
86 & s, const rclcpp::Time & stamp)
87 {
88     geometry_msgs::msg::PoseStamped ps;
89     ps.header.stamp = stamp;
90     ps.header.frame_id = "odom";
91     ps.pose.position.x = s.x;
92     ps.pose.position.y = s.y;

```

```

86     ps.pose.orientation.z = std::sin(s.theta / 2.0);
87     ps.pose.orientation.w = std::cos(s.theta / 2.0);
88     path.poses.push_back(ps);
89 }
90
91 double wheelbase_{1.5};
92 double dt_{0.05};
93 double sim_time_{0.0};
94 double noise_level_{0.0};
95
96 RobotState truth_state_, noisy_state_;
97 nav_msgs::msg::Path truth_path_, noisy_path_;
98
99 std::default_random_engine generator_;
100 std::normal_distribution<double> noise_dist_;
101
102 rclcpp::Publisher<nav_msgs::msg::Path>::SharedPtr
truth_path_pub_, noisy_path_pub_;
103 rclcpp::TimerBase::SharedPtr timer_;
104 };
105
106 int main(int argc, char ** argv)
107 {
108     rclcpp::init(argc, argv);
109     rclcpp::spin(std::make_shared<AckermannNoiseNode>());
110     rclcpp::shutdown();
111     return 0;
112 }
113 }

```

3.3.4. Predicted Results

The table below shows the expected final position error (Euclidean distance to ground truth) after 10 seconds of simulation:

Noise σ	Observation
0.001	Nearly invisible in RViz
0.01	Visibly drifts from truth
1.0	Large divergence

The simulation video is uploaded [here](#).

3.3.5. Discussion

Q: How does odometry drift accumulate?

Each integration step adds a small heading error $\Delta\theta$. Because $\dot{x} = v \cos \theta$ and $\dot{y} = v \sin \theta$, any error in θ propagates into both position components at every future step. The result is that position error grows faster than linearly — turning makes it worse because steering noise directly increases $\Delta\omega$ and thus $\Delta\theta$.

Q: Which noise source (velocity or steering) causes more drift?

Steering encoder noise is generally more damaging. An error in φ changes the yaw rate ω , which rotates the robot's heading. Once the heading is wrong, every future position update is in the wrong direction. Velocity noise only scales the distance traveled and does not rotate the trajectory.

References

- [1] ROS2 Development Team, “Managed Nodes (Lifecycle),” https://design.ros2.org/articles/node_lifecycle.html
- [2] ROS2 API Reference, `rclcpp_lifecycle::LifecycleNode`, https://docs.ros2.org/humble/api/rclcpp_lifecycle
- [3] ros2_controllers, “Mobile Robot Kinematics,” https://control.ros.org/humble/doc/ros2_controllers/doc/mobile_robot_kinematics.html
- [4] ROS2 TF2 Tutorials, <https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Tf2-Main.html>
- [5] Open Robotics, “ROS2 Documentation (Humble),” <https://docs.ros.org/en/humble>
- [6] Siegwart, R., Nourbakhsh, I.R., Scaramuzza, D., *Introduction to Autonomous Mobile Robots*, 2nd ed., MIT Press, 2011.